

## Summary of regular-expression constructs

| Construct                                      | Matches                                                                                 |
|------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Characters</b>                              |                                                                                         |
| <code>x</code>                                 | The character <code>x</code>                                                            |
| <code>\\</code>                                | The backslash character                                                                 |
| <code>\0n</code>                               | The character with octal value <code>0n</code> ( $0 \leq n \leq 7$ )                    |
| <code>\0nn</code>                              | The character with octal value <code>0nn</code> ( $0 \leq n \leq 7$ )                   |
| <code>\0mnn</code>                             | The character with octal value <code>0mnn</code> ( $0 \leq m \leq 3, 0 \leq n \leq 7$ ) |
| <code>\xhh</code>                              | The character with hexadecimal value <code>0xhh</code>                                  |
| <code>\uhhhh</code>                            | The character with hexadecimal value <code>0xhhhh</code>                                |
| <code>\t</code>                                | The tab character ( <code>'\u0009'</code> )                                             |
| <code>\n</code>                                | The newline (line feed) character ( <code>'\u000A'</code> )                             |
| <code>\r</code>                                | The carriage-return character ( <code>'\u000D'</code> )                                 |
| <code>\f</code>                                | The form-feed character ( <code>'\u000C'</code> )                                       |
| <code>\a</code>                                | The alert (bell) character ( <code>'\u0007'</code> )                                    |
| <code>\e</code>                                | The escape character ( <code>'\u001B'</code> )                                          |
| <code>\cx</code>                               | The control character corresponding to <code>x</code>                                   |
| <b>Character classes</b>                       |                                                                                         |
| <code>[abc]</code>                             | a, b, or c (simple class)                                                               |
| <code>[^abc]</code>                            | Any character except a, b, or c (negation)                                              |
| <code>[a-zA-Z]</code>                          | a through z or A through Z, inclusive (range)                                           |
| <code>[a-d[m-p]]</code>                        | a through d, or m through p: <code>[a-dm-p]</code> (union)                              |
| <code>[a-z&amp;&amp;[def]]</code>              | d, e, or f (intersection)                                                               |
| <code>[a-z&amp;&amp;[^bc]]</code>              | a through z, except for b and c: <code>[ad-z]</code> (subtraction)                      |
| <code>[a-z&amp;&amp;[^m-p]]</code>             | a through z, and not m through p: <code>[a-~q-z]</code> (subtraction)                   |
| <b>Predefined character classes</b>            |                                                                                         |
| <code>.</code>                                 | Any character (may or may not match <a href="#">line terminators</a> )                  |
| <code>\d</code>                                | A digit: <code>[0-9]</code>                                                             |
| <code>\D</code>                                | A non-digit: <code>^[0-9]</code>                                                        |
| <code>\s</code>                                | A whitespace character: <code>[ \t\n\x0B\f\r]</code>                                    |
| <code>\S</code>                                | A non-whitespace character: <code>^[ \t\n\x0B\f\r]</code>                               |
| <code>\w</code>                                | A word character: <code>[a-zA-Z_0-9]</code>                                             |
| <code>\W</code>                                | A non-word character: <code>^[a-zA-Z_0-9]</code>                                        |
| <b>POSIX character classes (US-ASCII only)</b> |                                                                                         |
| <code>\p{Lower}</code>                         | A lower-case alphabetic character: <code>[a-z]</code>                                   |
| <code>\p{Upper}</code>                         | An upper-case alphabetic character: <code>[A-Z]</code>                                  |

|                         |                                                                              |
|-------------------------|------------------------------------------------------------------------------|
| <code>\p{ASCII}</code>  | All ASCII: <code>[\x00-\x7F]</code>                                          |
| <code>\p{Alpha}</code>  | An alphabetic character: <code>[\p{Lower}\p{Upper}]</code>                   |
| <code>\p{Digit}</code>  | A decimal digit: <code>[0-9]</code>                                          |
| <code>\p{Alnum}</code>  | An alphanumeric character: <code>[\p{Alpha}\p{Digit}]</code>                 |
| <code>\p{Punct}</code>  | Punctuation: One of <code>!"#\$%&amp;'()*+,-./:;&lt;=&gt;?@[\]^_`{ }~</code> |
| <code>\p{Graph}</code>  | A visible character: <code>[\p{Alnum}\p{Punct}]</code>                       |
| <code>\p{Print}</code>  | A printable character: <code>[\p{Graph}]</code>                              |
| <code>\p{Blank}</code>  | A space or a tab: <code>[\t]</code>                                          |
| <code>\p{Cntrl}</code>  | A control character: <code>[\x00-\x1F\x7F]</code>                            |
| <code>\p{XDigit}</code> | A hexadecimal digit: <code>[0-9a-fA-F]</code>                                |
| <code>\p{Space}</code>  | A whitespace character: <code>[\t\n\x0B\f\r]</code>                          |

### Classes for Unicode blocks and categories

|                                         |                                                                |
|-----------------------------------------|----------------------------------------------------------------|
| <code>\p{InGreek}</code>                | A character in the Greek block (simple <a href="#">block</a> ) |
| <code>\p{Lu}</code>                     | An uppercase letter (simple <a href="#">category</a> )         |
| <code>\p{Sc}</code>                     | A currency symbol                                              |
| <code>\P{InGreek}</code>                | Any character except one in the Greek block (negation)         |
| <code>[\p{L}&amp;&amp;[^\p{Lu}]]</code> | Any letter except an uppercase letter (subtraction)            |

### Boundary matchers

|                 |                                                                            |
|-----------------|----------------------------------------------------------------------------|
| <code>^</code>  | The beginning of a line                                                    |
| <code>\$</code> | The end of a line                                                          |
| <code>\b</code> | A word boundary                                                            |
| <code>\B</code> | A non-word boundary                                                        |
| <code>\A</code> | The beginning of the input                                                 |
| <code>\G</code> | The end of the previous match                                              |
| <code>\Z</code> | The end of the input but for the final <a href="#">terminator</a> , if any |
| <code>\z</code> | The end of the input                                                       |

### Greedy quantifiers

|                     |                                                           |
|---------------------|-----------------------------------------------------------|
| <code>X?</code>     | <code>X</code> , once or not at all                       |
| <code>X*</code>     | <code>X</code> , zero or more times                       |
| <code>X+</code>     | <code>X</code> , one or more times                        |
| <code>X{n}</code>   | <code>X</code> , exactly $n$ times                        |
| <code>X{n,}</code>  | <code>X</code> , at least $n$ times                       |
| <code>X{n,m}</code> | <code>X</code> , at least $n$ but not more than $m$ times |

### Reluctant quantifiers

|                  |                                     |
|------------------|-------------------------------------|
| <code>X??</code> | <code>X</code> , once or not at all |
| <code>X*?</code> | <code>X</code> , zero or more times |
| <code>X+?</code> | <code>X</code> , one or more times  |

|              |                                                |
|--------------|------------------------------------------------|
| $X\{n\}?$    | $X$ , exactly $n$ times                        |
| $X\{n, \}$   | $X$ , at least $n$ times                       |
| $X\{n, m\}?$ | $X$ , at least $n$ but not more than $m$ times |

### Possessive quantifiers

|              |                                                |
|--------------|------------------------------------------------|
| $X?+$        | $X$ , once or not at all                       |
| $X^*+$       | $X$ , zero or more times                       |
| $X++$        | $X$ , one or more times                        |
| $X\{n\}+$    | $X$ , exactly $n$ times                        |
| $X\{n, \}+$  | $X$ , at least $n$ times                       |
| $X\{n, m\}+$ | $X$ , at least $n$ but not more than $m$ times |

### Logical operators

|       |                                            |
|-------|--------------------------------------------|
| $XY$  | $X$ followed by $Y$                        |
| $X Y$ | Either $X$ or $Y$                          |
| $(X)$ | $X$ , as a <a href="#">capturing group</a> |

### Back references

|                |                                                                      |
|----------------|----------------------------------------------------------------------|
| $\backslash n$ | Whatever the $n^{\text{th}}$ <a href="#">capturing group</a> matched |
|----------------|----------------------------------------------------------------------|

### Quotation

|                |                                                         |
|----------------|---------------------------------------------------------|
| $\backslash$   | Nothing, but quotes the following character             |
| $\backslash Q$ | Nothing, but quotes all characters until $\backslash E$ |
| $\backslash E$ | Nothing, but ends quoting started by $\backslash Q$     |

### Special constructs (non-capturing)

|                     |                                                                              |
|---------------------|------------------------------------------------------------------------------|
| $(?:X)$             | $X$ , as a non-capturing group                                               |
| $(?idsux-idmsux)$   | Nothing, but turns match flags on - off                                      |
| $(?idsux-idmsux:X)$ | $X$ , as a <a href="#">non-capturing group</a> with the given flags on - off |
| $(?=X)$             | $X$ , via zero-width positive lookahead                                      |
| $(?!X)$             | $X$ , via zero-width negative lookahead                                      |
| $(?<=X)$            | $X$ , via zero-width positive lookbehind                                     |
| $(?<!X)$            | $X$ , via zero-width negative lookbehind                                     |
| $(?>X)$             | $X$ , as an independent, non-capturing group                                 |

## Backslashes, escapes, and quoting

The backslash character (`'\'`) serves to introduce escaped constructs, as defined in the table above, as well as to quote characters that otherwise would be interpreted as unescaped constructs. Thus the expression `\\` matches a single backslash and `\{` matches a left brace.

It is an error to use a backslash prior to any alphabetic character that does not denote an escaped construct; these are reserved for future extensions to the regular-expression language. A backslash may be used prior to a non-alphabetic character regardless of whether that character is part of an unescaped construct.

Backslashes within string literals in Java source code are interpreted as required by the [Java Language Specification](#) as either [Unicode escapes](#) or other [character escapes](#). It is therefore necessary to double backslashes in string literals that represent regular expressions to protect them from interpretation by the Java bytecode compiler. The string literal `"\b"`, for example, matches a single backspace character when interpreted as a regular expression, while `"\\b"` matches a word boundary. The string literal `"\(\hello\"` is illegal and leads to a compile-time error; in order to match the string `(hello)` the string literal `"\\(\hello\\"` must be used.

## Character Classes

Character classes may appear within other character classes, and may be composed by the union operator (implicit) and the intersection operator (`&&`). The union operator denotes a class that contains every character that is in at least one of its operand classes. The intersection operator denotes a class that contains every character that is in both of its operand classes.

The precedence of character-class operators is as follows, from highest to lowest:

- |   |                |                                     |
|---|----------------|-------------------------------------|
| 1 | Literal escape | <code>\x</code>                     |
| 2 | Grouping       | <code>[...]</code>                  |
| 3 | Range          | <code>a-z</code>                    |
| 4 | Union          | <code>[a-e][i-u]</code>             |
| 5 | Intersection   | <code>[a-z&amp;&amp;[aeiou]]</code> |

Note that a different set of metacharacters are in effect inside a character class than outside a character class. For instance, the regular expression `.` loses its special meaning inside a character class, while the expression `-` becomes a range forming metacharacter.

## Line terminators

A *line terminator* is a one- or two-character sequence that marks the end of a line of the input character sequence. The following are recognized as line terminators:

- A newline (line feed) character (`'\n'`),
- A carriage-return character followed immediately by a newline character (`"\r\n"`),
- A standalone carriage-return character (`'\r'`),
- A next-line character (`'\u0085'`),
- A line-separator character (`'\u2028'`), or

- A paragraph-separator character ('`\u2029`).

If [UNIX\\_LINES](#) mode is activated, then the only line terminators recognized are newline characters.

The regular expression `.` matches any character except a line terminator unless the [DOTALL](#) flag is specified.

By default, the regular expressions `^` and `$` ignore line terminators and only match at the beginning and the end, respectively, of the entire input sequence. If [MULTILINE](#) mode is activated then `^` matches at the beginning of input and after any line terminator except at the end of input. When in [MULTILINE](#) mode `$` matches just before a line terminator or the end of the input sequence.

## Groups and capturing

Capturing groups are numbered by counting their opening parentheses from left to right. In the expression `((A)(B(C)))`, for example, there are four such groups:

- 1 `((A)(B(C)))`
- 2 `(A)`
- 3 `(B(C))`
- 4 `(C)`

Group zero always stands for the entire expression.

Capturing groups are so named because, during a match, each subsequence of the input sequence that matches such a group is saved. The captured subsequence may be used later in the expression, via a back reference, and may also be retrieved from the matcher once the match operation is complete.

The captured input associated with a group is always the subsequence that the group most recently matched. If a group is evaluated a second time because of quantification then its previously-captured value, if any, will be retained if the second evaluation fails.

Matching the string "aba" against the expression `(a(b)?)+`, for example, leaves group two set to "b". All captured input is discarded at the beginning of each match.

Groups beginning with `(?` are pure, *non-capturing* groups that do not capture text and do not count towards the group total.

## Unicode support

This class follows [Unicode Technical Report #18: Unicode Regular Expression Guidelines](#), implementing its second level of support though with a slightly different concrete syntax.

Unicode escape sequences such as `\u2014` in Java source code are processed as described in [?3.3](#) of the Java Language Specification. Such escape sequences are also implemented directly by the regular-expression parser so that Unicode escapes can be used in expressions that are read from files or from the keyboard. Thus the strings `"\u2014"` and `"\\u2014"`, while not equal, compile into the same pattern, which matches the character with hexadecimal value `0x2014`.

Unicode blocks and categories are written with the `\p` and `\P` constructs as in Perl. `\p{prop}` matches if the input has the property *prop*, while `\P{prop}` does not match if the input has that property. Blocks are specified with the prefix `In`, as in `InMongolian`. Categories may be specified with the optional prefix `Is`: Both `\p{L}` and `\p{IsL}` denote the category of Unicode letters. Blocks and categories can be used both inside and outside of a character class.

The supported blocks and categories are those of [The Unicode Standard, Version 3.0](#). The block names are those defined in Chapter 14 and in the file [Blocks-3.txt](#) of the [Unicode Character Database](#) except that the spaces are removed; "Basic Latin", for example, becomes "BasicLatin". The category names are those defined in table 4-5 of the Standard (p. 88), both normative and informative.

## Comparison to Perl 5

Perl constructs not supported by this class:

- The conditional constructs `(?{X})` and `(?(condition)X|Y)`,
- The embedded code constructs `(?{code})` and `(??{code})`,
- The embedded comment syntax `(?#comment)`, and
- The preprocessing operations `\l \u`, `\L`, and `\U`.

Constructs supported by this class but not by Perl:

- Possessive quantifiers, which greedily match as much as they can and do not back off, even when doing so would allow the overall match to succeed.
- Character-class union and intersection as described [above](#).

Notable differences from Perl:

- In Perl, `\1` through `\9` are always interpreted as back references; a backslash-escaped number greater than 9 is treated as a back reference if at least that many subexpressions exist, otherwise it is interpreted, if possible, as an octal escape. In this class octal escapes must always begin with a zero. In this class, `\1` through `\9` are always interpreted as back references, and a larger number is accepted as a back reference if at least that many subexpressions exist at that point in the regular expression, otherwise the parser will drop digits until the number is smaller or equal to the existing number of groups or it is one digit.

- Perl uses the `g` flag to request a match that resumes where the last match left off. This functionality is provided implicitly by the [Matcher](#) class: Repeated invocations of the [find](#) method will resume where the last match left off, unless the matcher is reset.
- In Perl, embedded flags at the top level of an expression affect the whole expression. In this class, embedded flags always take effect at the point at which they appear, whether they are at the top level or within a group; in the latter case, flags are restored at the end of the group just as in Perl.
- Perl is forgiving about malformed matching constructs, as in the expression `*a`, as well as dangling brackets, as in the expression `abc]`, and treats them as literals. This class also accepts dangling brackets but is strict about dangling metacharacters like `+`, `?` and `*`, and will throw a [PatternSyntaxException](#) if it encounters them.